Research Agent

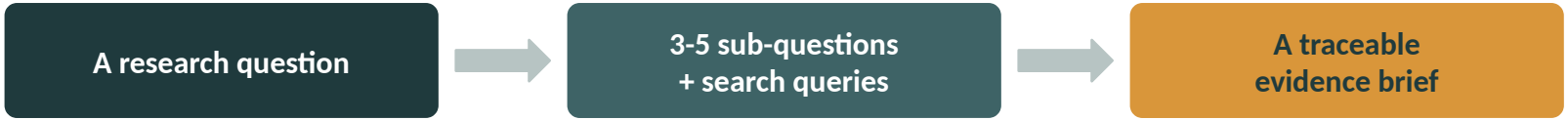
An LLM-powered planning agent for academic literature search

Individual project presentation

Intelligent Agents (PCOM7E), University of Essex Online · October 2026

`github.com/sergio88mi/research-agent`

The brief, and the design I was implementing



From the Group D design proposal (Group D, 2026) - full references in the README and the transcript

What the design fixed in advance

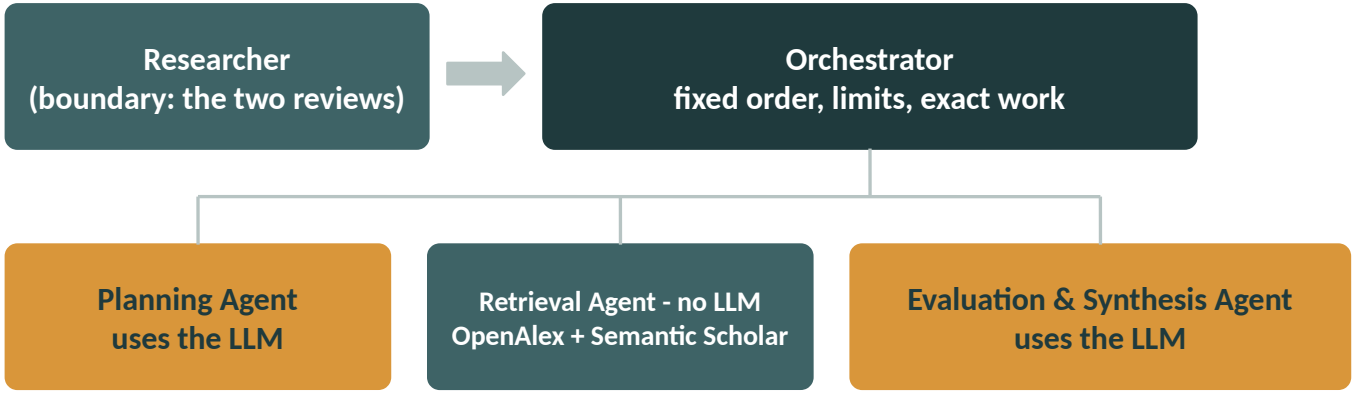
- Three specialised agents and a deterministic coordinator (Hong et al., 2024; Kim et al., 2026)
- Two human review points: the plan, then the evidence (Zhu et al., 2026)
- Every DOI checked against Crossref before it can be cited (Walters and Wilder, 2023)
- Bounded autonomy: one query retry per sub-question, one scope revision per run
- The brief must say what it could not cover

The failure it was designed against

A language model asked for literature will produce plausible papers that do not exist. So the model never writes a reference: it only plans queries, scores and summarises. Everything with an exact answer - de-duplication, DOI checks, the reference list - is ordinary code.

My task: build the proposed system on my own, test it, show it running - and, once the feedback on the design arrived (read 29 Sept), act on its three points.

Architecture: three agents, one coordinator



Data passed between them: SubQuestion · Paper · Assessment · Brief (the classes from Diagram 1, as Pydantic models)

- Decomposing, scoring and summarising are judgement tasks - the model does those
- Counting, de-duplicating, verifying DOIs, applying cut-offs - plain code does those
- Every model reply is validated against a schema; one repair retry, then the run stops
- Limits, model name and the list of sources live in one configuration file
- Agents by role, each with one goal (Wooldridge and Jennings, 1995; Wooldridge, Jennings and Kinny, 2000); one coordinator, because errors spread without central verification (Kim et al., 2026)

3-5
sub-questions

1
query retry per sub-question

1
scope revision per run

3 / 5
relevance cut-off

How one run flows

1 Plan	Review 1	2 Retrieve	3 Verify	4 Score	Review 2	5 Synthesise
question → 3-5 sub-questions with search queries	approve, or describe the change → re-plan	OpenAlex + Semantic Scholar, 20 records each; failover	de-duplicate by DOI; check each DOI in Crossref	relevance 1-5 with a reason, from the abstract	approve · drop papers · reject scope (once)	notes, themes with citations, gaps, limitations

One command starts it: `python main.py "What approaches reduce hallucination in LLM-based autonomous agents?"`

If a search falls short - or a source fails

- Fewer than 5 usable records, or fewer than 5 relevant after scoring: the Planning Agent rewrites the query once, told why the search fell short - never twice
- A source that fails after its retries is skipped for that query and tried again on the next; the brief records it

Output

- brief.md for reading, brief.json for machines
- Numbered papers; notes, themes and references share the numbering
- run.log records every decision the agents made

Demo 1 - planning and the first review

```
=== Review 1: proposed research plan ===
Interpretation: This research question investigates architectural, algorithmic, and evaluation techniques used to mitigate ungrounded or incor
rect outputs generated by large language model-driven autonomous agents.

  1. What retrieval-augmented generation and external tool-integration frameworks reduce hallucination in autonomous LLM agents?
     query: retrieval augmented generation tool integration reduce hallucination llm agents
  2. How do self-reflection, planning feedback loops, and multi-agent consensus mechanisms mitigate unfaithful reasoning?
     query: self reflection multi agent collaboration reduce hallucination autonomous reasoning
  3. Which fine-tuning, alignment, and constrained decoding methods specifically target hallucination reduction in agentic workflows?
     query: reinforcement learning alignment fine tuning decoding hallucination reduction agents
  4. How are hallucinations in autonomous agents benchmarked, detected, and evaluated during long-horizon tasks?
     query: benchmarking detecting hallucination evaluation metrics autonomous language model agents

Type 'approve' to search, or describe what to change (e.g. 'focus on healthcare, drop #3').
>
```

Screenshot: Review 1 on 15 Sept, on gemini-3.6-flash. The Lite model later planned the same question differently (first box).

The Lite model's plan

On the Lite model (23 Sept) the four sub-questions became: prompting and reasoning · retrieval and tools · self-correction and memory · evaluation. The 30 Sept run on the next two slides reused that plan.

Shown before anything is searched

The researcher sees exactly what will be searched. Nothing has been searched yet, so the review loop is unlimited; each revision costs one model request and produces a new plan.

Read again on 1 Oct

The revision prompt lacked the plan under review, so 'merge 3 and 4' just dropped the old third sub-question. Fixed: on 2 Oct (FT-12) 1 and 2 stayed word for word; 3 and 4 were combined, minus 'memory'.

Demo 2 - two sources, verify, score, retry once

```
# agents/orchestrator.py, _gather_evidence() - excerpt, comments as in the file
# Trigger 1 - the Retrieval Agent's own goal check: enough usable records?
if not self.retriever.assess_threshold(found) and self._may_reformulate(sq):
    sq = self._reformulate(plan, idx, self.retriever.reformulation_reason(found), question)
    found += self.retriever.retrieve_evidence(sq)
new = validation.validate_dois_and_metadata(self._novel(found, seen))
assessed = self.evaluator.evaluate_evidence(question, [sq], new)
# Trigger 2 - the orchestrator's check after screening: enough *relevant* records?
# Stage 4's live run showed why this matters: sub-question 4's search returned 20
# abstracts, 12 of them new, and only one answered it. Same single-retry budget.
relevant = sum(1 for a in assessed if a.selected)
if relevant < config.RETRIEVAL_THRESHOLD and self._may_reformulate(sq):
    ...   # same rewrite, then search and score the new query
```

```
# run.log, 30 Sept 10:11-10:15 (abridged: no timestamps, ... marks a cut)
retrieval | retrieving | subquestion=1 | ... | sources=openalex,semanticscholar
retrieval | retrieved  | subquestion=1 | source=openalex | records=20 | with_abstract=20
http      | semanticscholar | HTTP 429 (transient) - retry 4/4 in 24s
retrieval | subquestion=1 | source=semanticscholar failed (... after 4 retries
            (last problem: HTTP 429)) - continuing with remaining sources
orchestr. | threshold | subquestion=4 | the query returned 8 records but only 2 were relevant
            (minimum 5); ... | reformulating (1 of 1 allowed)
retrieval | retrieved | subquestion=4 | sources_answered=openalex,semanticscholar | records=40 ...
orchestr. | threshold | subquestion=4 | after reformulation: 33 new records, 20 relevant
```

Above: the code that decides on the one query rewrite, as submitted. Below: run.log, 30 Sept - a source failing over, and that rewrite with both sources answering.

140 → 117

records from 2 sources → unique papers

78 / 117

papers with a DOI verified in Crossref; the rest flagged, never dropped

77 kept

of 117 at cut-off 3/5, each with a reason

2 → 22

relevant papers for sub-question 4 after the one retry (2 → 6 with one source)

Demo 3 - the second review and the brief

```
# terminal, 30 Sept 10:15-10:18 (abridged; ... marks a cut)
=== Review 2: selected evidence ===
Sub-question 4: How are hallucination rates and factual accuracy evaluated
  in multi-step agent workflows? (query reformulated once: ... )
  65. [5/5] AgentHallu: Benchmarking Automated Hallucination Attribution of
            LLM-based Agents (2026) - DOI UNVERIFIED
  76. [5/5] From Output Errors to Workflow Harm: A Practitioner-Audit Method
            for LLM-Mediated Research (2026) - DOI verified
77 papers selected; 40 screened out (reasons are recorded in the brief).
Type 'approve' to write the brief, or 'drop 3, 7' ..., or 'reject' ...
> approve
evaluation   | summarising | subquestion=4 | batch=1/3 | papers=10
evaluation   | summarised | papers=77
evaluation   | synthesised | themes=4 | gaps=3
117 unique papers across 4 sub-questions | 78 DOIs verified
  | 77 approved, 40 excluded | 4 themes, 3 gaps
  | reformulations: 1, scope revisions: 0
Saved: output/brief-20260930-101857-what-approaches-...md
```

The reference list is built from the records' own metadata - the model cites papers by number and never writes a reference.

From the brief of 30 Sept (... marks a cut)

Evidence note [76] The paper presents TRACE, a practitioner-audit framework ... demonstrated through an empirical analysis of 45 documentation-positive incidents recorded by a clinician-informatician. ...
Checked: the figure 45 is in the paper's abstract. The paper came from Semantic Scholar only.

Theme
Evaluation of multi-step agent workflows involves specialized benchmarks, automated hallucination attribution frameworks, multi-agent audits, and domain-aware metrics to track trajectory-level and cross-source errors. [63, 64, 65, 67, 73, 74, 76, 77]

Gap
Few papers address the computational overhead and latency introduced by continuous self-correction loops ...

Limitation
... semanticscholar: 2 of 5 searches answered, 40 records. ... Automatic failover was used: semanticscholar failed for 3 search(es) ...

Testing: three layers of evidence

113

unit tests

pytest, no network. Recorded API replies in tests/fixtures. Data models, abstract reconstruction, de-duplication and DOI checks, the 3-5 rule, schema validation and repair, review loops, threshold and retry caps, cut-off, batching, themes citing non-existent papers, retry policies, brief layout; the second source and failover; the cache, logging, error messages and how a run ends.

16

functional tests

Live, against the real services, with steps, expected and observed result. FT-01 to FT-11, 17-30 Sept: missing key, drop, revise, reject scope, full run, threshold retry, transient errors, quota stop, JSON repair, two sources with failover. FT-12 to FT-16, 2 Oct, on the submitted code: revise with the fix, three other questions, stop messages. All pass, two with a qualification (FT-03, FT-14). Plus an offline replay of the 30 Sept run.

21

remediation entries

Every problem met during development, its cause, the change made, and how the fix was verified - the model retirement, 503s, timeouts, quotas, OpenAlex's policy change, a typo that re-planned; and ten found on 1 Oct by reading the code and the documents against each other.

```
pytest -> 113 passed     (full output: evidence/pytest_113_passed.txt)
per file:  cache 6 · evaluation 13 · http 9 · llm_client 5 · logging_setup 2 · main 12 · models 5 · planning 10 · retrieval 7 ·
s2_client 5 · search_client 6 · stage5 14 · storage 2 · synthesis 9 · validation 8
second model on FT-13 -> would keep 27 of 35 kept (clear answers) · κ 0.65 · agreement, not precision  (evidence/model_judge_*)
```

What went wrong, and what changed

When	What happened	What changed
14 Sept	First Gemini call: HTTP 404 - the model the build was first configured for had been retired by the provider	Model name was already in configuration: a change of setting, no change to the logic
17 Sept	Run died mid-scoring on 503, then on a read timeout	Transient-error retry with back-off; 'busy' and 'wrong' errors treated differently
17-23 Sept	429 - a 20-requests-a-day free quota, burned by retries	Quota id logged; per-day limit stops at once; switched to a 500-a-day model
23 Sept	OpenAlex 429: keyless access now capped at $0.10/day	Free API key added to configuration; one shared retry policy for OpenAlex and Crossref
23 Sept	The typo 'approv' was taken as feedback and re-planned	Short unrecognised input is re-asked instead of sent to the planner

None of these needed a rewrite: settings in one module and each service behind one small client kept the fixes small, and the second source was one new module. Commits start on 9 Sept, about one per stage, the tests growing from 5 to 68; a read-through on 1 Oct added ten more entries (remediation log #12-#21) and took the tests to 113; five live tests on 2 Oct ran the corrected code.

Against the design, and honestly

Matches the design

- Three agents, orchestrator, the data classes and Decision enum of Diagram 1 (field changes listed in the README)
- Both review points, both autonomy caps
- Crossref verification; flagged, never dropped
- Brief with notes, themes, gaps, limitations, references
- Departures: OpenAlex first, not Semantic Scholar; checks run per sub-question, with a second retry trigger after scoring; Crossref compares titles only; a second scope rejection is refused, not a stop; one test question until the last day, not a varied set used throughout

Changed after the feedback

- Model choice justified from the provider's figures (Google, 2026b, 2026c, 2026d; Google DeepMind, 2026a): 1M-token context vs prompts of at most ~12k tokens, structured output (function calling is not used), 500 requests/day free, $0.30 per M input tokens, benchmarks - none of which measures abstract screening
- Targets and datasets stated: precision ≥ 0.8 on a gold set, recall ≥ 0.6 against a published review's list - neither built yet, so not measured; de-duplication missed (4 pairs in 77); one run checked by a second model: κ 0.65, agreement, not precision
- Built: two sources for every search, automatic failover; 77 → 117 unique papers on the same plan and model

Limits and next steps

- Abstracts only, 20 per query per source
- Lite kept 42 of 77 papers, Flash 26 of 78 (each on its own plan) - calibrate the cut-off per model
- A paper under two DOIs is not merged; Semantic Scholar's keyless pool refused 3 of 5 searches on 30 Sept and all 16 live ones on 2 Oct
- One research question during development; three more run once each on 2 Oct, their briefs not judged
- Free-tier prompts may be used by the provider: a confidential question needs a paid key
- Next: DataCite lookup, the gold set, full text where open

`github.com/sergio88mi/research-agent · README: setup, keys, how to run, references`